

Sample Project Report



Military College of Signals
National University of Science and
Technology (NUST)
Department of Computer Software
Engineering



PROJECT TITLE

Course Code: CS-251

Course Title: Design and Analysis of Algorithm

Name Student ID

Project Supervisor: Dr. Samra Kanwal

Submission Date :

Project Report: Fraud Detection in Online Food Orders

1. Introduction

Fraud in online food ordering systems is a growing concern, leading to financial losses and operational inefficiencies. This project aims to detect fraudulent orders using efficient algorithmic approaches in the Design and Analysis of Algorithms (DAA) framework.

2. Problem Statement

Given a dataset of online food orders containing order details such as location, payment method, frequency, and order amount, the objective is to develop an algorithm that detects fraudulent orders with high accuracy and efficiency.

3. Objective

The fraud detection system employs a combination of **K-Means Clustering** (for anomaly detection), **Random Forest** (for classification), and **Graph-Based Anomaly Detection** (for pattern analysis in user behavior).

4. Methodology

4.1 Algorithm Description and Implementation

Algorithm 2 Fraud Detection in Online Food Orders

1: Load dataset D with n orders	
2: Preprocess dataset: Remove duplicates, normalize, encode categorical variables	▷ $O(n)$
3: Apply K-Means clustering with k clusters	▷ $O(nki)$
4: for each order o_i in D do	
5: Compute distance from cluster centroid	▷ $O(d)$
6: if o_i is an outlier then	
7: Mark as potential fraud	▷ $O(1)$
8: end if	
9: end for	
10: Train Random Forest on training data	▷ $O(m \log m)$
11: Construct transaction graph G	▷ $O(V + E)$
12: for each user node v in G do	
13: Perform BFS/DFS to detect anomalies	▷ $O(V + E)$
14: end for	

Step 1: Data Preprocessing

- Remove duplicate orders
- Standardize numerical values (e.g., order amounts, timestamps)
- Encode categorical variables (e.g., payment method, order location)

Step 2: Anomaly Detection using K-Means Clustering

- Partition orders into **k** clusters based on feature similarity.
- Identify outliers by calculating the distance of each order from the cluster centroid.
- Orders far from centroids are marked as potential frauds.

Time Complexity: $O(n * k * i * d)$

Where:

- n = number of orders,
- k = number of clusters,
- i = number of iterations,
- d = number of dimensions (features).

Space Complexity: $O(n * d)$

(Each order with d features must be stored.)

Step 3: Classification using Random Forest

- Train a **Random Forest** model using labeled fraud and non-fraud orders.
- Predict the fraud probability for new orders.

Time Complexity: $O(m * \log m)$ per tree

Where:

- m = number of training samples.

Space Complexity: $O(m * f)$

(f = number of features used in splitting nodes.)

Step 4: Graph-Based Anomaly Detection

- Construct a graph where nodes represent users and edges represent transaction links.
- Identify suspicious patterns based on centrality, connected components, and abrupt changes in user behavior.

Time Complexity: $O(V + E)$ (Breadth-First Search for anomaly detection)

(V = users, E = transactions.)

Space Complexity: $O(V + E)$

(Adjacency list representation of the graph.)

5. Big-O Notation Summary

Step	Algorithm	Time Complexity	Space Complexity
Data Preprocessing	Cleaning & Encoding	$O(n)$	$O(n)$
Anomaly Detection	K-Means Clustering	$O(n * k * i * d)$	$O(n * d)$
Classification	Random Forest	$O(m * \log m)$	$O(m * f)$
Pattern Analysis	Graph Search	$O(V + E)$	$O(V + E)$

6. Conclusion

The proposed fraud detection system combines multiple algorithmic techniques to ensure high accuracy and efficiency. K-Means identifies outliers, Random Forest classifies orders, and Graph-Based Anomaly Detection detects unusual user behavior. The approach is scalable and can be optimized for real-time fraud detection.

7. Future Enhancements

- Implement **Deep Learning (LSTMs)** for sequential fraud detection.
- Improve clustering by using **DBSCAN** for non-spherical fraud patterns.
- Deploy the system in a real-time environment with low-latency processing.